

Лекція 1. Вступ до вебтехнологій та архітектури Мережі

Предмет і завдання курсу

Головна мета цього курсу — перетворити сприйняття Всесвітньої мережі як споживача на професійне інженерне розуміння її внутрішніх механізмів. Сучасний веброзробник — це не просто фахівець, який пише код мовами HTML чи JavaScript, а інженер, який розуміє повний цикл існування розподіленої мережевої системи: від фізичної передачі бітів через мережеве обладнання до складної логіки обробки даних на сервері та рендерингу пікселів у браузері користувача.

Навчальний план курсу розроблено за принципом від простого до складного та розбито на послідовний цикл лекційних і практичних занять. У межах лекційного блоку ми розглянемо:

- Мережеві протоколи та архітектурні шаблони;
- Мови розмітки та стилізації для створення вебінтерфейсів;
- Клієнтське програмування та асинхронну взаємодію;
- Серверну розробку та роботу з базами даних.

На практичних заняттях застосуємо отримані знання для створення власних проектів. Підсумковий контроль передбачає захист лабораторного практикуму.

Еволюція Всесвітньої павутини (Web 1.0, Web 2.0, Web 3.0)

Щоб зрозуміти сучасний стан веброзробки, необхідно розглянути історію еволюції технологій.

ЕВОЛЮЦІЯ ВЕБУ	
Web 1.0 (Static Web) - Статичний HTML/CSS - Пасивне споживання	Read-Only - Односторонній контент - Сервер видає готові файли
Web 2.0 (Dynamic Web) - Динамічний JS, AJAX - SPA, REST API, Social	Read-Write - Користувацький контент (UGC) - Двостороння взаємодія
Web 3.0 (Semantic/Decent) - Децентралізація, AI - Smart Contracts, Crypto	Read-Write-Own - Семантичні дані (RDF/OWL) - Машиночитаний контент

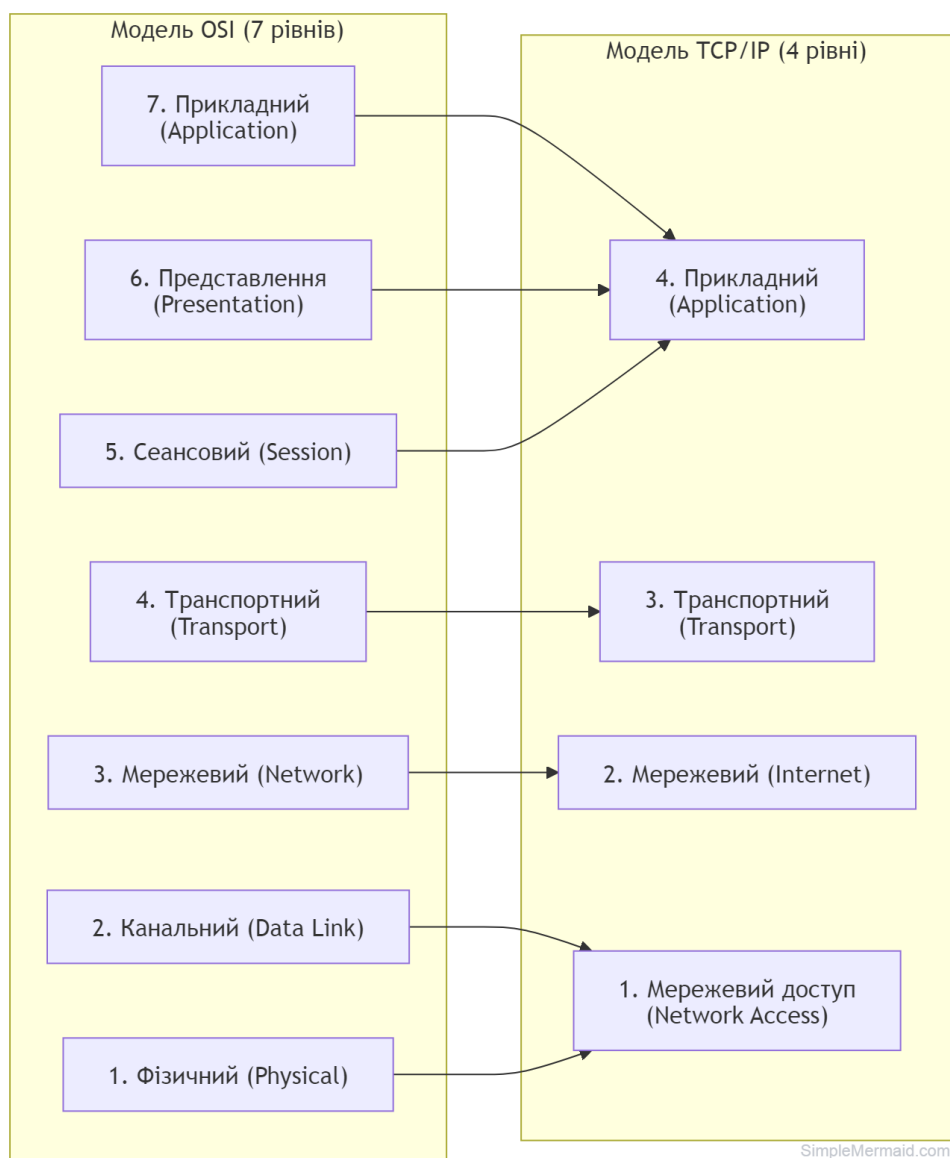
1. **Web 1.0 («Read-Only Web», 1990-х — початок 2000-х років).** Тісно пов'язаний з ім'ям Тіма Бернерса-Лі, який у 1989 році в CERN запропонував концепцію гіпертекстових документів. Веб цього періоду складався зі статичних HTML-сторінок, поєднаних гіперпосиланнями. Користувачі були пасивними споживачами інформації. Сервер лише віддавав заздалегідь підготовлені текстові та графічні файли.
2. **Web 2.0 («Read-Write Web» або «Social Web», з середини 2000-х років).** Поява технології AJAX (Asynchronous JavaScript and XML), спрощення роботи з DOM та виникнення засобів для створення динамічного контенту змінили парадигму. Користувач перетворився з пасивного читача на співавтора контенту (User-Generated Content). Почалася ера соціальних мереж, блог-платформ, відеохостингів та складних вебзастосунків (Single Page Applications, SPA). На серверній стороні стали популярними динамічні мови програмування (PHP, Python, Ruby, Java, пізніше Node.js) та СУБД.
3. **Web 3.0 («Read-Write-Own Web» / Semantic Web / Decentralized Web).** Сучасний етап розвитку, що базується на двох концепціях:

- **Семантичний веб:** Структурування даних у форматі, придатному для автоматизованого оброблення інтелектуальними агентами та алгоритмами ІІІ (стандарти RDF, OWL, мікророзмітка Schema.org).
- **Децентралізований веб:** Використання блокчейн-технологій, криптографії та розподілених файлових систем (IPFS) для забезпечення автономності користувачів та захисту їхніх персональних даних.

Мережева інфраструктура та адресація

Моделі мережевої взаємодії: OSI та TCP/IP

Передача даних у мережі Інтернет здійснюється за чітко визначеними правилами — протоколами. Для опису мережевої взаємодії використовуються дві концептуальні моделі: еталонна модель OSI та практична модель TCP/IP.



Еталонна модель OSI (Open Systems Interconnection)

Складається з 7 рівнів:

1. **Фізичний (Physical):** Передача сирих бітів через фізичне середовище (електричні імпульси, світловод, радіохвилі).

2. **Канальний (Data Link):** Адресація на основі MAC-адрес, формування кадрів (frames), виявлення помилок (Ethernet, Wi-Fi).
3. **Мережевий (Network):** Маршрутизація пакетів між різними мережами за допомогою IP-адрес (IP, ICMP).
4. **Транспортний (Transport):** Надійна або швидка доставка даних між кінцевими вузлами (TCP, UDP).
5. **Сеансовий (Session):** Встановлення, підтримка та завершення сеансів зв'язку.
6. **Представлення (Presentation):** Форматування, шифрування, стиснення даних (SSL/TLS, ASCII, JPEG).
7. **Прикладний (Application):** Інтерфейс між мережевими службами та користувацькими програмами (HTTP, FTP, SMTP, SSH).

Модель TCP/IP

Практична модель, на якій побудовано Інтернет. Вона об'єднує верхні рівні OSI:

1. **Рівень мережевого доступу (Network Access):** Об'єднує Фізичний та Канальний рівні OSI.
2. **Мережевий рівень (Internet Layer):** Відповідає за IP-адресацію та маршрутизацію.
3. **Транспортний рівень (Transport Layer):** Забезпечує зв'язок між процесами за допомогою портів.
4. **Прикладний рівень (Application Layer):** Об'єднує 5-й, 6-й та 7-й рівні OSI. Саме тут працює протокол HTTP/HTTPS, який є основним об'єктом вивчення у веброзробці.

Ідентифікація ресурсів у мережі

Для того, щоб два комп'ютери могли обмінюватися даними, їм потрібні унікальні адреси.

IP-адресація

IPv4 (Internet Protocol version 4): 32-бітна адреса, що записується у вигляді чотирьох десяткових чисел, розділених крапками (наприклад, 192.168.1.1 або 142.250.180.14). Забезпечує близько 2^{32} - 4.3 мільярда адрес. Через вичерпання IPv4-адрес активно впроваджується **IPv6**.

IPv6 (Internet Protocol version 6): 128-бітна адреса, що записується у шістнадцятковій формі та розділяється двокрапками (наприклад, 2001:0db8:85a3:0000:0000:8a2e:0370:7334). Забезпечує 2^{128} адрес.

Адреси поділяються на:

- **Публічні (Public IP):** Унікальні в межах всього Інтернету.
- **Приватні (Private IP):** Використовуються у локальних мережах (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16).
- **Loopback (Локальна петля):** 127.0.0.1 (або localhost), що вказує на поточний пристрій.

Мережеві порти

IP-адреса ідентифікує конкретний вузол (пристрій) у мережі. Проте на цьому пристрої одночасно може працювати безліч мережесх програм. Для адресації конкретного процесу використовуються **порти** — 16-бітні числа (від 0 до 65535).

- **Системні (Well-Known Ports, 0–1023):** Зарезервовані для стандартних служб (HTTP — 80, HTTPS — 443, SSH — 22, FTP — 21, DNS — 53).
- **Зареєстровані (Registered Ports, 1024–49151):** Використовуються для користувацьких серверних процесів (наприклад, Express.js часто використовує 3000, PostgreSQL — 5432, MongoDB — 27017).
- **Динамічні / Ефемерні (Dynamic/Private Ports, 49152–65535):** Виділяються операційною системою клієнта тимчасово для вихідного з'єднання.

Система доменних імен (DNS)

Людям важко запам'ятовувати числові IP-адреси. Для трансляції людино-читаних доменних імен (наприклад, `example.com`) у числові IP-адреси використовується децентралізована розподілена база даних — **DNS (Domain Name System)**.

Ієрархія доменних імен

1. **Кореневий домен (Root domain):** Позначається крапкою наприкінці (зазвичай випускається): `example.com.`
2. **Домени верхнього рівня (Top-Level Domains, TLD):**
 - *gTLD (загальні):* `.com`, `.org`, `.net`, `.edu`.
 - *ccTLD (національні):* `.ua`, `.uk`, `.de`, `.us`.
3. **Домени другого рівня (Second-Level Domain, SLD):** Наприклад, `google` у `google.com`.
4. **Субдомени (Subdomains):** Наприклад, `api.google.com` або `mail.google.com`.

Алгоритм DNS-резолвінгу (пошуку IP-адреси)

Коли користувач вводить у браузері `[https://example.com]` (`https://example.com`), відбувається наступне:

1. **Локальний кеш:** Браузер перевіряє власний кеш, потім кеш операційної системи (файли `hosts` та кеш DNS-клієнта OS).
2. **DNS Resolver (Рекурсивний резолвер):** Якщо IP не знайдено, запит надсилається на рекурсивний DNS-сервер провайдера або публічний DNS (наприклад, `8.8.8.8` від Google чи `1.1.1.1` від Cloudflare).
3. **Кореневі сервери (Root Name Servers):** Резолвер запитує один із 13 масивів корневих серверів. Кореневий сервер повертає адресу TLD-сервера, відповідального за зону `.com`.
4. **TLD-сервери (Top-Level Domain Servers):** Резолвер звертається до TLD-сервера `zone .com`, який повертає адресу **авторитетного (Authoritative) DNS-сервера**, що обслуговує домен `example.com`.
5. **Авторитетний DNS-сервер:** Повертає безпосередню IP-адресу (наприклад, `93.184.216.34`), записану в A-реєстрі домену.
6. **Кешування:** Резолвер та операційна система збережуть отриманий результат у своєму кеші на час, визначений параметром **TTL (Time To Live)**.

Анатомія URL / URI / URN

- **URI (Uniform Resource Identifier):** Загальний термін для ідентифікації ресурсів.
- **URN (Uniform Resource Name):** Ідентифікує ресурс за назвою у певному просторі імен, але не вказує, де він знаходиться (наприклад, `urn:isbn:978-3-16-148410-0`).

- **URL (Uniform Resource Locator):** Найпоширеніший тип URI, який не тільки ідентифікує ресурс, але й вказує спосіб його досягнення (протокол) та місцезнаходження.

Повна структура URL

Розглянемо детальний приклад:

`https://admin:p@ssword@www.example.com:8080/path/to/resource?category=web&sort=asc#section-2`

1. **Схема / Протокол (`https://`):** Визначає протокол доступу до ресурсу (`http`, `https`, `ftp`).
2. **Автентифікація (`admin:p@ssword@`):** Облікові дані користувача (зараз практично не використовуються з міркувань безпеки).
3. **Хост / Доменне ім'я (`[www.example.com]` (`https://www.example.com`)):** Домен або IP-адреса сервера.
4. **Порт (`:8080`):** Мережевий порт. Якщо випущений, використовуються замовчувані значення (80 для HTTP, 443 для HTTPS).
5. **Шлях до ресурсу (`/path/to/resource`):** Логічний шлях на сервері до конкретної сторінки, файлу або ендпоінту API.
6. **Параметри запити / Query String (`?category=web&sort=asc`):** Набір пар `ключ=значення`, відокремлених амперсандом (`&`). Використовуються для фільтрації, сортування, передачі невеликих даних.
7. **Фрагмент / Якір / Anchor (`#section-2`):** Вказує на конкретну секцію всередині документа. Не надсилається на сервер, обробляється виключно браузером.

Клієнт-серверна архітектура

Базові концепції та розподіл відповідальності

Фундаментальною архітектурною моделлю Інтернету є **клієнт-серверна система**. Це концепція мережевої взаємодії, у якій ролі розподілені між постачальниками послуг (серверами) та їх замовниками (клієнтами).

Клієнт (Client-Side / Frontend)

Клієнт — це ініціатор мережевого запиту. У веброзробці основними клієнтами є:

- **Веббраузери:** Chrome, Firefox, Safari, Edge (виконують рендеринг HTML/CSS та виконання коду JavaScript).
- **Мобільні та десктопні застосунки:** Взаємодіють з сервером через HTTP/HTTPS API.
- **CLI-інструменти та скрипти:** `curl`, `Postman`, автоматизовані боти, скрейпери.

Основні завдання клієнта: Відображення користувацького інтерфейсу (UI), обробка дій користувача (кліки, введення тексту), первинна валідація даних, відправлення запитів на сервер та візуалізація отриманих даних.

Сервер (Server-Side / Backend)

Сервер — це програмно-апаратний комплекс, який перебуває в очікуванні запитів від клієнтів, обробляє їх, виконує необхідну бізнес-логіку та повертає відповідь.

Основні завдання сервера: Збереження та обробка даних, робота з базами даних (СУБД), автентифікація та авторизація користувачів, виконання складних обчислень, забезпечення безпеки даних, інтеграція зі сторонніми сервісами.

Повний життєвий цикл HTTP-запиту/відповіді (Request-Response Cycle)

Коли користувач відкриває браузер і вводить у стрічку адреси (`https://my-app.com/dashboard`), операційна система та мережевий стек здійснюють цілий комплекс послідовних дій.

Етап 1: Розпізнавання та DNS-резолвінг

Браузер розбирає введений URL, виділяє протокол (`https`), хост (`my-app.com`), порт (443 за замовчуванням) та шлях (`/dashboard`). Далі виконується перевірка кешів та надсилання DNS-запиту для визначення IP-адреси хоста.

Етап 2: Встановлення транспортного з'єднання (TCP 3-Way Handshake)

Перед передачею даних за протоколом HTTP необхідно встановити надійне транспортне з'єднання за допомогою протоколу **TCP**. Це відбувається у три кроки («триетапне рукошлякування»):

1. **SYN (Synchronize):** Клієнт надсилає серверу пакет із прапором `SYN` та випадковим початковим порядковим номером (`Sequence\,Number = X`).
2. **SYN-ACK (Synchronize-Acknowledge):** Сервер приймає пакет, відповідає пакетом із прапорами `SYN` та `ACK`. Вказує свій початковий номер (`Sequence\,Number = Y`) та підтверджує отримання номера клієнта (`Acknowledgment\,Number = X + 1`).
3. **ACK (Acknowledge):** Клієнт надсилає підтвердження з прапором `ACK` (`Sequence\,Number = X + 1, Acknowledgment\,Number = Y + 1`). TCP-з'єднання встановлено.

Етап 3: Встановлення захищеного з'єднання (TLS/SSL Handshake)

Оскільки використовується протокол `https`, поверх TCP розгортається захищений шар TLS:

1. Клієнт і сервер узгоджують версію протоколу TLS та алгоритми шифрування (`Cipher Suites`).
2. Сервер надає клієнту свій цифровий **SSL/TLS сертифікат**.
3. Клієнт перевіряє сертифікат через центр сертифікації (CA).
4. За допомогою асиметричного шифрування обмінюються ключем, після чого створюється спільний симетричний **сесійний ключ шифрування**. Усі подальші дані шифруються цим ключем.

Етап 4: Передача HTTP-запиту (HTTP Request)

Клієнт формує текстовий (або бінарний у випадках HTTP/2, HTTP/3) HTTP-запит і надсилає його шифрованим каналом. Запит містить метод (`GET`), шлях (`/dashboard`), заголовки (`Host`, `User-Agent`, `Cookie` тощо).

Етап 5: Обробка запиту сервером

1. Вебсервер (наприклад, Nginx) приймає TCP-пакет, розшифровує його та передає застосунку (наприклад, Express.js / Node.js).

2. Застосунок перевіряє маршрут (Routing), виконує проміжні обробники (Middleware: автентифікація, перевірка прав).
3. Виконується бізнес-логіка: звернення до бази даних (наприклад, PostgreSQL чи MongoDB), обробка отриманих записів.
4. Застосунок формує підсумковий контент (HTML-сторінку, JSON-об'єкт, зображення тощо) та встановлює відповідні HTTP-заголовки.

Етап 6: Надсилання HTTP-відповіді (HTTP Response)

Сервер надсилає відповідь клієнту, що містить рядок статусу (наприклад, HTTP/1.1 200 OK), заголовки відповідей (Content-Type, Set-Cookie) та тіло відповіді (HTML, CSS, JS або JSON).

Етап 7: Завершення або збереження з'єднання

Залежно від заголовка Connection (keep-alive чи close), TCP-з'єднання або залишається відкритим для наступних запитів (наприклад, завантаження зображень, стилів, скриптів), або закривається за допомогою процедури TCP 4-Way Handshake.

Протокол HTTP/HTTPS

Принципи та еволюція протоколу HTTP

HTTP (HyperText Transfer Protocol) — це протокол прикладного рівня, який є основною «мовою» обміну даними в Інтернеті.

Ключові характеристики HTTP:

1. **Stateless (Без збереження стану):** Сервер не зберігає інформацію про попередні запити того самого клієнта між окремими транзакціями. Кожен запит є незалежним. Для підтримки стану (наприклад, авторизованого сеансу) використовуються додаткові механізми: Cookies, Sessions, JWT (JSON Web Tokens).
2. **Extensible (Розширюваність):** Завдяки кастомним HTTP-заголовкам можна легко додавати нову функціональність без зміни самого протоколу.
3. **Media Independent (Незалежність від типу даних):** HTTP може передавати будь-які типи даних, якщо клієнт і сервер знають, як їх обробляти (зазначено в заголовку Content-Type).

Еволюція протоколу HTTP

- **HTTP/1.0 (1996):** На кожен окремий запит (наприклад, завантаження 10 картинок) відкривалося окреме TCP-з'єднання, яке закривалося одразу після передачі даних. Це створювало величезне навантаження на мережу.
- **HTTP/1.1 (1997/2014):** Впроваджено підтримку довготривалих з'єднань (Keep-Alive). Одне TCP-з'єднання могло використовуватися для послідовного надсилання кількох запитів. Додано конвеєризацію (Pipelining), заголовок Host (що дозволило віртуальний хостинг кількох сайтів на одній IP-адресі) та потокову передачу даних (Chunked transfer encoding).
- **HTTP/2 (2015):** Перехід від текстового до **бінарного** формату. Впроваджено **мультиплексування**: можливість надсилати десятки запитів та відповідей паралельно через одне-єдине TCP-з'єднання у вигляді окремих кадрів (frames) і потоків (streams). Це повністю усунуло проблему Head-of-Line (HoL) Blocking на рівні HTTP.

- **HTTP/3 (2022):** Заміна транспортного протоколу TCP на протокол **QUIC** (який працює поверх **UDP**). Це дозволило вирішити проблему HoL Blocking на транспортному рівні (коли втрата одного пакета TCP зупиняє передачу всіх паралельних потоків) та прискорити встановлення з'єднання (0-RTT/1-RTT handshake).

Структура HTTP-запиту (Request)

HTTP-запит — це текстове (у разі HTTP/1.1) або бінарне (HTTP/2, HTTP/3) повідомлення, яке складається з трьох основних частин:

1. **Рядок старту (Start Line / Request Line);**
2. **Заголовки запиту (Headers);**
3. **Тіло запиту (Body) — необов'язкове.**

```
POST /api/users HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Content-Type: application/json
Content-Length: 45
Authorization: Bearer eyJhbGciOiJIUzI1Ni...

{
  "username": "john_doe",
  "email": "john@example.com"
}
```

Анатомія рядка старту запиту:

```
POST /api/users HTTP/1.1
```

- **POST** — HTTP-метод (дієслово), що вказує дію;
- **/api/users** — Шлях (URI) ресурсу;
- **HTTP/1.1** — Версія протоколу.

Методи HTTP (HTTP Methods / Verbs)

Метод HTTP вказує на бажану дію, яку необхідно виконати над ресурсом.

Метод	Опис	Ідемпотентність	Safe (Безпечний)	Наявність Тіла (Body)
GET	Отримання даних з сервера.	Так	Так	Ні
POST	Створення нового ресурсу або відправка даних на обробку.	Ні	Ні	Так

Метод	Опис	Ідемпотентність	Safe (Безпечний)	Наявність Тіла (Body)
PUT	Повне оновлення ресурсу (або створення, якщо не існує).	Так	Ні	Так
PATCH	Часткове оновлення ресурсу.	Ні	Ні	Так
DELETE	Видалення ресурсу.	Так	Ні	Необов'язково
HEAD	Аналогічний GET, але повертає тільки заголовки без тіла.	Так	Так	Ні
OPTIONS	Запитує підтримувані методи та параметри для ресурсу (CORS).	Так	Так	Ні

Важливі концепції методів:

- **Безпечний метод (Safe Method):** Метод, який не змінює стан сервера (не створює побічних ефектів). GET та HEAD є безпечними, оскільки вони призначені лише для читання.
- **Ідемпотентний метод (Idempotent Method):** Метод, повторне виконання якого з тими самими параметрами дає точно такий самий результат на сервері, як і одноразове виконання.
 - *Приклад:* PUT /users/42 з даними {"name": "Alice"} — скільки б разів ви не надсилали цей запит, стан ресурсу із ID 42 залишиться однаковим (name = "Alice"). Метод є ідемпотентним.
 - *Приклад:* POST /users з даними {"name": "Alice"} — кожен новий запит створюватиме *нового* користувача у базі даних (ID 43, ID 44 і так далі). Метод НЕ є ідемпотентним.

Структура HTTP-відповіді (Response)

HTTP-відповідь має схожу на запит структуру та складається з:

1. **Рядка статусу (Status Line);**
2. **Заголовків відповіді (Headers);**
3. **Тіла відповіді (Body) — необов'язкове.**

```
HTTP/1.1 200 OK
Date: Wed, 02 Sep 2026 08:30:00 GMT
Server: Nginx/1.24.0
Content-Type: application/json; charset=UTF-8
Content-Length: 53
Set-Cookie: session_id=abc123xyz; Secure; HttpOnly
```

```
{
  "id": 42,
  "status": "success",
  "message": "User created"
}
```

Анатомія рядка статусу:

HTTP/1.1 200 OK

- HTTP/1.1 — Версія протоколу;
- 200 — Числовий код стану (Status Code);
- OK — Текстове пояснення (Reason Phrase).

Коди відповідей HTTP (Status Codes)

Коди відповідей розбиті на 5 класів за першою цифрою:

1xx: Informational (Інформаційні)

Вказують на те, що запит прийнято і процес триває.

- 100 Continue: Сервер прийняв заголовки запиту, клієнт може продовжувати надсилання тіла.
- 101 Switching Protocols: Сервер погоджується змінити протокол (наприклад, перехід з HTTP на WebSocket).

2xx: Success (Успішна обробка)

Запит був успішно отриманий, зрозумілий та прийнятий сервером.

- 200 OK: Стандартна відповідь на успішний запит (GET, PUT, POST).
- 201 Created: Запит виконано успішно, і в результаті було створено новий ресурс (зазвичай у відповідь на POST).
- 204 No Content: Сервер успішно обробив запит, але не повертає жодного тіла (часто для DELETE).

3xx: Redirection (Перенаправлення)

Клієнт повинен виконати додаткові дії для завершення запиту.

- 301 Moved Permanently: Запитуваний ресурс назавжди змінив свій URI. Нову адресу вказано в заголовку Location.
- 302 Found (Temporary Redirect): Ресурс тимчасово перебуває за іншим URI.

- 304 Not Modified: Ресурс не змінювався з часу останнього запиту (використовується для кешування). Тіло відповіді відсутнє.

4xx: Client Errors (Помилки клієнта)

Запит містить невірний синтаксис або не може бути виконаний з вини клієнта.

- 400 Bad Request: Запит синтаксично некоректний або сервер не може його зрозуміти.
- 401 Unauthorized: Для отримання доступу потрібна автентифікація (відсутній або недійсний токен/пароль).
- 403 Forbidden: Клієнт автентифікований, але не має прав доступу до даного ресурсу.
- 404 Not Found: Запитуваний ресурс не знайдено на сервері.
- 405 Method Not Allowed: HTTP-метод запиту не підтримується для даного ендпоінту (наприклад, спроба зробити POST для ресурсу, який підтримує лише GET).
- 422 Unprocessable Entity: Запит сформовано вірно, але він містить семантичні помилки (наприклад, не пройдено валідацію полів JSON).
- 429 Too Many Requests: Клієнт надіслав занадто багато запитів за проміжок часу (Rate Limiting).

5xx: Server Errors (Помилки сервера)

Сервер зазнав невдачі при спробі виконати коректно сформований запит.

- 500 Internal Server Error: Загальна помилка сервера (необроблена виняткова ситуація в коді).
- 502 Bad Gateway: Сервер, виконуючи роль мережевого шлюзу чи проксі, отримав недійсну відповідь від вищестоящего сервера.
- 503 Service Unavailable: Сервер тимчасово не здатен обробляти запити (перевантажений або перебуває на обслуговуванні).
- 504 Gateway Timeout: Шлюз не отримав відповідь від вищестоящего сервера у визначений таймаут.

Основні HTTP-заголовки (Headers)

Заголовки передають додаткову інформацію про запит або відповідь у формі пар Ключ : Значення.

1. **Загальні заголовки (General Headers):**
 - Date: Дата і час формування повідомлення.
 - Cache-Control: Директиви управління кешуванням (no-cache, max-age=3600).
2. **Заголовки запиту (Request Headers):**
 - Host: Доменне ім'я та порт сервера (обов'язковий у HTTP/1.1).
 - User-Agent: Рядок, що ідентифікує браузер/операційну систему клієнта.
 - Accept: Типи контенту, які клієнт здатний обробити (text/html, application/json).
 - Authorization: Облікові дані для автентифікації (Bearer <token>).
 - Cookie: Сесійні куки, що передаються на сервер.
3. **Заголовки відповіді (Response Headers):**
 - Server: Інформація про програмне забезпечення вебсервера (Nginx, Apache).
 - Set-Cookie: Передає куки від сервера до клієнта для збереження у браузері.
 - Location: Вказує URI при перенаправленні (для кодів 3xx або 201).
4. **Заголовки сутності / вмісту (Entity Headers):**

- Content-Type: MIME-тип даних у тілі повідомлення (text/html, application/json, image/png).
- Content-Length: Розмір тіла у байтах.

Захищений протокол HTTPS

HTTPS (HyperText Transfer Protocol Secure) — це розширення протоколу HTTP, у якому всі дані, що передаються між клієнтом та сервером, шифруються за допомогою протоколів **TLS (Transport Layer Security)** або його застарілого попередника **SSL (Secure Sockets Layer)**.

HTTPS гарантує три фундаментальні аспекти безпеки:

1. **Конфіденційність (Encryption):** Шифрування даних унеможливорює перехоплення прочитаної інформації третіми особами (захист від Man-in-the-Middle атак).
2. **Цілісність даних (Data Integrity):** Будь-яка спроба модифікації даних під час передачі буде виявлена за допомогою контрольних сум (MAC).
3. **Автентифікація (Authentication):** Підтвердження того, що користувач взаємодіє саме з тим сервером, до якого звертався, за допомогою **цифрових сертифікатів SSL/TLS**, виданих довіреними центрами сертифікації (Certificate Authorities, CA).

Як браузер рендерить вебсторінку

Після того, як браузер отримав від сервера HTTP-відповідь із кодом 200 OK та тілом у вигляді HTML-коду, починається складний процес перетворення тексту на інтерактивне графічне зображення на екрані користувача. Цей процес називається **Critical Rendering Path (Критичний шлях рендерингу)**.

Етапи Critical Rendering Path

1. Побудова DOM (Document Object Model)

Браузер читає сирі байти HTML-документа, декодує їх у символи відповідно до кодування (наприклад, UTF-8), виділяє токени (теги <p>, <div> тощо), перетворює токени на вузли (Nodes) і будує древовидну структуру — **DOM-дерево**. DOM представляє собою програмну модель документа, доступну для маніпуляцій з мови JavaScript.

2. Побудова CSSOM (CSS Object Model)

Коли браузер зустрічає теги <style> або зовнішні CSS-файли (<link rel="stylesheet">), він паралельно починає парсинг стилів та будує **CSSOM-дерево**. CSSOM містить інформацію про правила та стилі, які застосовуються до відповідних вузлів DOM.

3. Формування дерева рендерингу (Render Tree)

Браузер об'єднує DOM та CSSOM у **Render Tree**.

- Render Tree містить тільки **видимі** на сторінці елементи.
- Вузли, які мають стиль display: none;, теги <head>, <script>, <meta> **НЕ входять** до Render Tree.
- Елементи із visibility: hidden; входять до Render Tree, оскільки вони займають місце на екрані.

4. Обчислення макета (Layout / Reflow)

Браузер розраховує точні геометричні розміри та позиціонування кожного елемента Render Tree на екрані пристрою залежно від розміру вікна огляду (Viewport).

5. Отрисовка (Paint / Rasterizing)

Браузер перетворює елементи Render Tree на реальні пікселі на екрані. На цьому етапі раструються тексти, кольори, фони, тіні, межі елементів. Для оптимізації процес може розбиватися на кілька шарів (Layers).

6. Композиція (Compositing)

Якщо елементи малювалися на окремих шарах (наприклад, при використанні CSS3 3D-трансформацій чи елементів `<video>`), браузер об'єднує ці шари у правильному порядку з урахуванням z-index та виводить фінальну картинку на екран.

Вплив CSS та JavaScript на завантаження сторінки

- **CSS є ресурсом, що блокує рендеринг (Render-blocking):** Браузер не почне малювати сторінку (не побудує Render Tree), поки повністю не завантажить і не розпарсить усі CSS-файли.
- **JavaScript є ресурсом, що блокує парсинг HTML (Parser-blocking):** Коли HTML-парсер зустрічає тег `<script src="script.js">`, він зупиняє побудову DOM, завантажує скрипт з мережі, виконує його і лише після цього продовжує парсинг HTML.

Оптимізація завантаження скриптів:

Щоб уникнути блокування рендерингу сторінки, використовуються атрибути `async` та `defer`:

- **Без атрибутів:** Парсинг HTML зупиняється. Скрипт завантажується та виконується негайно.
- **async (Асинхронно):** Скрипт завантажується фоново, не блокуючи парсинг HTML. Проте як тільки скрипт завантажено, парсинг HTML **зупиняється** для виконання скрипту. Порядок виконання кількох `async`-скриптів не гарантується (хто перший завантажився, той і виконується).
- **defer (Відкладено):** Скрипт завантажується фоново. Його виконання відбувається **тільки після того**, як HTML-документ повністю розпарсено (перед подією `DOMContentLoaded`). Гарантується порядок виконання скриптів відповідно до їх розташування в коді.

Підсумки

Сьогодні ми розглянули фундаментальні основи, на яких базується весь сучасний Інтернет та веброзробка:

1. Зрозуміли еволюцію Вебу від статичних сторінок Web 1.0 до динамічних застосунків Web 2.0 та децентралізованого Web 3.0.
2. Розглянули мережеві моделі OSI та TCP/IP, дізналися про роль IP-адрес, портів та систему доменних імен DNS.
3. Детально вивчили клієнт-серверну архітектуру та повний життєвий цикл обробки HTTP-запиту.

4. Проаналізували структуру HTTP-запитів та відповідей, класи кодів станів, методи HTTP, концепції безпеки HTTPS.
5. Розглянули критичний шлях рендерингу вебсторінки у браузері та дізналися, як правильно оптимізувати підключення ресурсів CSS та JavaScript.